



Universidade de Brasília
Departamento de Ciência da Computação

Algoritmos de Ordenação

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Definição

- ▶ A ordenação permite que o acesso aos dados seja feito de forma mais eficiente;
- ▶ Um algoritmo de ordenação é aquele que coloca os elementos de uma dada sequência em uma certa ordem predefinida;
- ▶ Algoritmos básicos de ordenação:
 - ✓ Ordenação por Bolha - *Bubble Sort*
 - ✓ Ordenação por Seleção - *Selection Sort*
 - ✓ Ordenação por Inserção - *Insertion Sort*
- ▶ Algoritmos sofisticados de ordenação
 - ✓ Algoritmo *Merge Sort*
 - ✓ Algoritmo *Quick Sort*
 - ✓ Algoritmo *Heap Sort*

Ordenação por Bolha

Bubble Sort

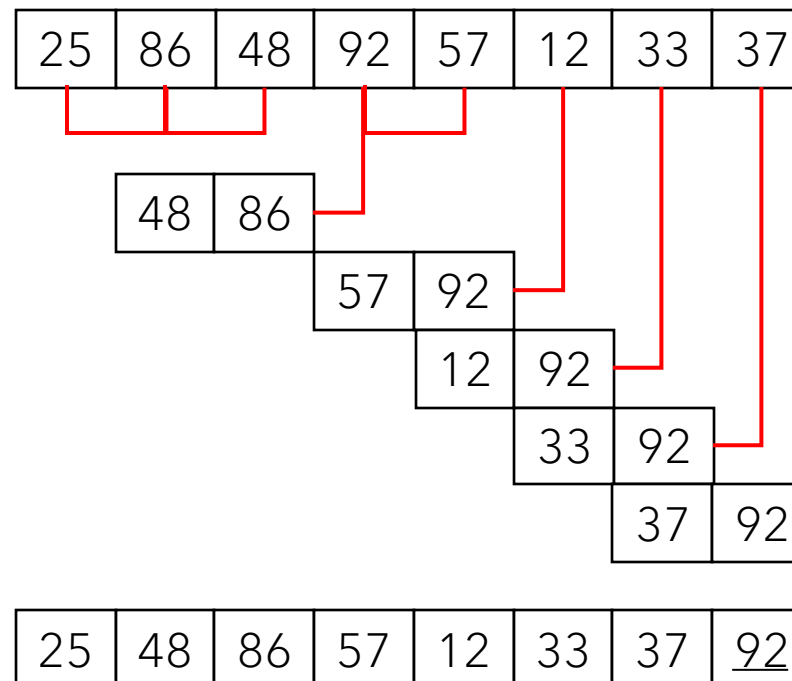
Ordenação por Bolha - *Bubble Sort*

- ▶ O princípio de funcionamento desse algoritmo é a troca de valores em posições consecutivas de *array* para que, desse modo, fiquem na ordem desejada;
- ▶ É um algoritmo simples, de fácil entendimento e implementação;
- ▶ Não é um algoritmo eficiente, estudado apenas para fins de desenvolvimento de raciocínio.

Início do passo: $i = 7$

⋮

Início do passo: $i = 0$



Ordenação por Bolha - *Bubble Sort*

```
void bubbleSort(int v[], int n){
    int i, continua, aux, fim = n;
    do{
        continua = 0;
        for(i = 0; i < fim - 1; i++){
            if(v[i] > v[i+1]){
                aux = v[i];
                v[i] = v[i+1];
                v[i+1] = aux;
                continua = i;
            }
        }
        fim--;
    }while (continua != 0);
}
```

Ordenação por Seleção

Selection Sort

Ordenação por Seleção - *Selection Sort*

- ▶ O algoritmo divide o *array* em duas partes:
 - ✓ A parte ordenada: à esquerda do elemento analisado;
 - ✓ A parte não ordenada: à direita do elemento.
- ▶ Para cada elemento do *array*, começando do primeiro, o algoritmo procura na parte não ordenada (direita) o menor valor (ordenação crescente) e troca os dois valores de lugar;
- ▶ Avança para a próxima posição do *array* e repete esse processo;
- ▶ Isso é feito até que todo *array* esteja ordenado;
- ▶ Não é um algoritmo eficiente.

Ordenação por Seleção - *Selection Sort*

$i = 0$	0	1	2	3	4	5	6	7
	25	86	48	92	57	12	33	37
$i = 1$	0	1	2	3	4	5	6	7
	12	86	48	92	57	25	33	37
$i = 2$	0	1	2	3	4	5	6	7
	12	25	48	92	57	86	33	37
$i = 3$	0	1	2	3	4	5	6	7
	12	25	33	92	57	86	48	37
	0	1	2	3	4	5	6	7
	12	25	33	37	57	86	48	92

$i = 5$	0	1	2	3	4	5	6	7
	12	25	33	37	57	86	48	92
$i = 6$	0	1	2	3	4	5	6	7
	12	25	33	37	48	86	57	92
$i = 7$	0	1	2	3	4	5	6	7
	12	25	33	37	48	57	86	92
	0	1	2	3	4	5	6	7
	12	25	33	37	48	57	86	92

Ordenação por Seleção - *Selection Sort*

```
void selectionSort(int v[], int n){
    int i, j, menor, troca;
    for(i = 0; i < n-1; i++){
        menor = i;
        for(j = i + 1; j < n; j++){
            if(v[j] < v[menor]){
                menor = j;
            }
        }
        if(i != menor){
            troca = v[i];
            v[i] = v[menor];
            v[menor] = troca;
        }
    }
}
```

Ordenação por Inserção

Insertion Sort

Ordenação por Inserção - *Insertion Sort*

- ▶ Tem esse nome pois se assemelha ao processo de ordenação de um conjunto de cartas de baralhos com as mãos: pega-se uma carta de cada vez e a “insere” em seu devido lugar, sempre deixando as cartas da mão em ordem;
- ▶ O algoritmo percorre um *array* e, para cada posição **X**, verifica se o seu valor está na posição correta;
- ▶ Isso é feito andando para o começo do *array* a partir da posição **X** e movimentando uma posição para frente os valores que são maiores do que o valor da posição **X**;
- ▶ Desse modo, teremos uma posição livre para inserir o valor da posição **X** em seu devido lugar.

Ordenação por Inserção - *Insertion Sort*

25	86	48	92	57	12	33	37
----	----	----	----	----	----	----	----

$i = 1$

0	1	2	3	4	5	6	7
25	86	48	92	57	12	33	37
25	86	48	92	57	12	33	37

$i = 2$

0	1	2	3	4	5	6	7
25	86	48	92	57	12	33	37
25	48	86	92	57	12	33	37

$i = 3$

0	1	2	3	4	5	6	7
25	48	86	92	57	12	33	37
25	48	86	92	57	12	33	37

0	1	2	3	4	5	6	7
25	48	86	92	57	12	33	37

$i = 4$

25	48	57	86	92	12	33	37
----	----	----	----	----	----	----	----

0	1	2	3	4	5	6	7
25	48	57	86	92	12	33	37

$i = 5$

12	25	48	57	86	92	33	37
----	----	----	----	----	----	----	----

0	1	2	3	4	5	6	7
12	25	48	57	86	92	33	37

$i = 6$

12	25	33	48	57	86	92	37
----	----	----	----	----	----	----	----

$i = 7$

0	1	2	3	4	5	6	7
12	25	33	48	57	86	92	37
12	25	33	37	48	57	86	92

Ordenação por Inserção - *Insertion Sort*

```
void insertionSort(int v[], int n){  
    int i, j, atual;  
    for(i = 1; i < n; i++){  
        atual = v[i];  
        for(j = i; (j > 0) && (atual < v[j-1]); j--){  
            v[j] = v[j - 1];  
        }  
        v[j] = atual;  
    }  
}
```

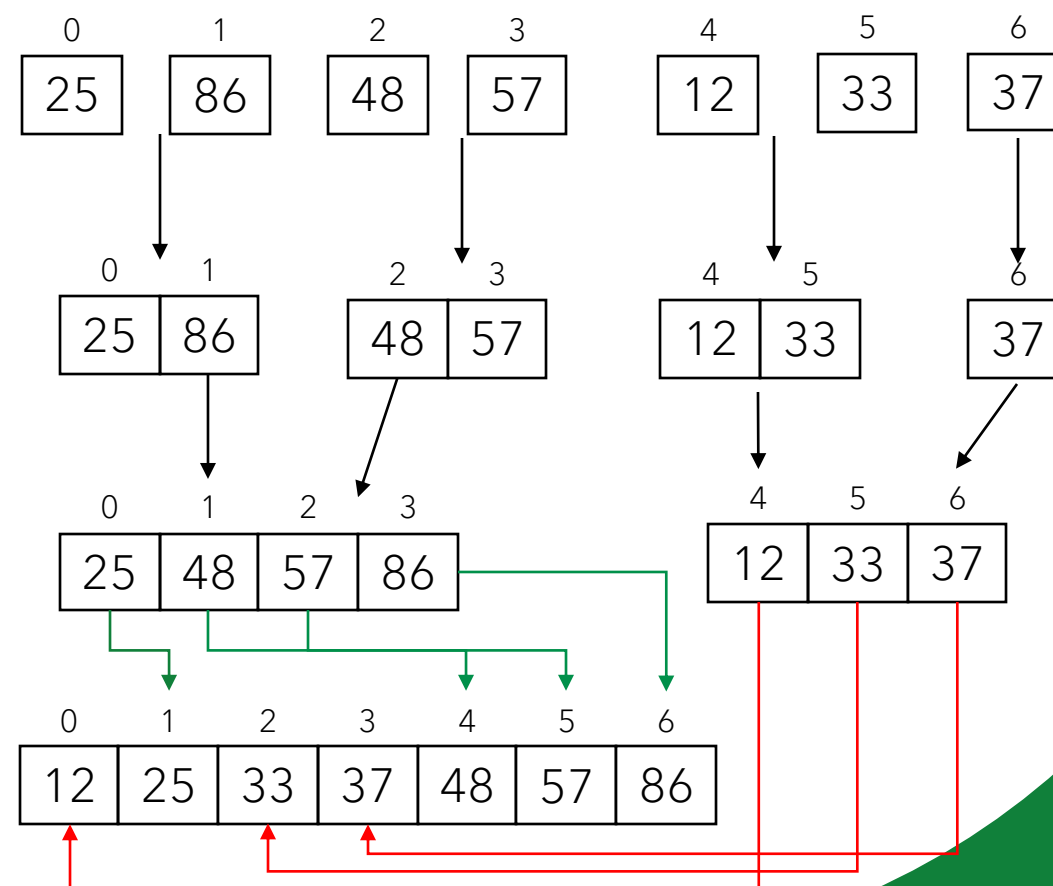
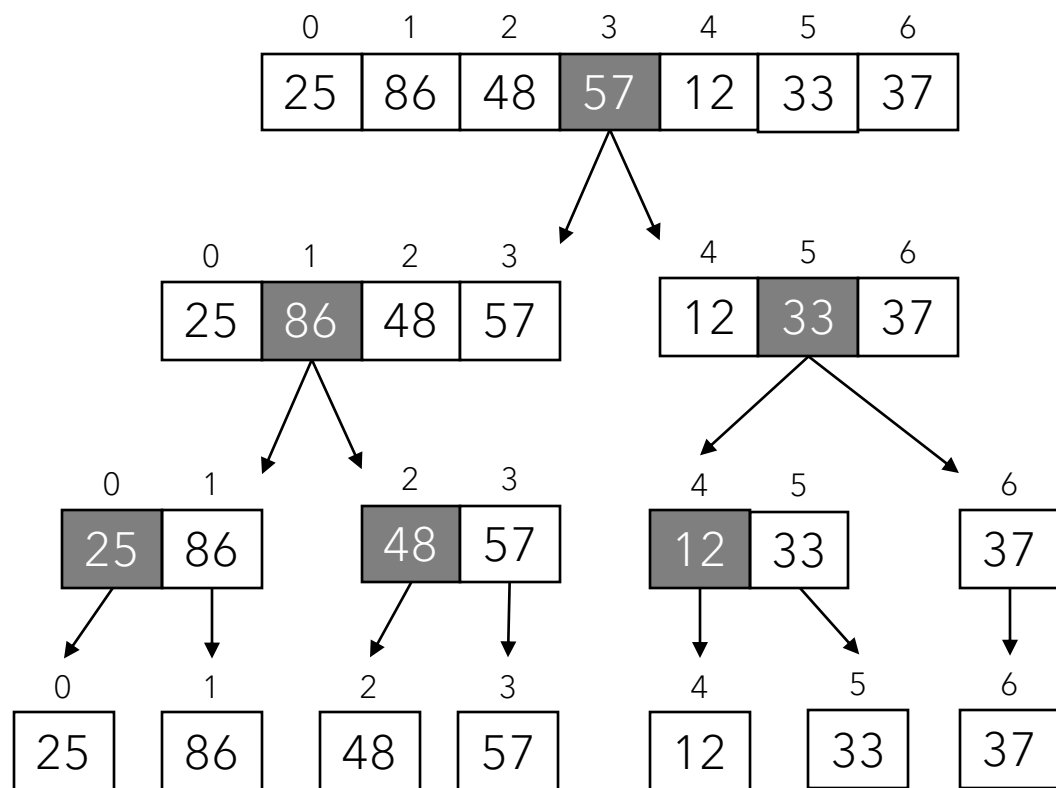
Algoritmo Merge Sort

Algoritmo *Merge Sort*

- ▶ É conhecido como ordenação por “intercalação”;
- ▶ É um algoritmo recursivo que usa a ideia de “dividir para conquistar” para ordenar os dados de um *array*;
- ▶ O algoritmo divide, recursivamente, o *array* em duas partes até que cada posição dele seja considerada como um *array* de um único elemento;
- ▶ Em seguida, o algoritmo combina dois *arrays* de forma a obter um *array* maior e ordenado;
- ▶ Essa combinação dos *arrays* é feita intercalando seus elementos de acordo com o sentido da ordenação (crescente ou decrescente);
- ▶ Esse processo se repete até que exista apenas um *array*.

Algoritmo Merge Sort

0	1	2	3	4	5	6
25	86	48	57	12	33	37



Algoritmo Merge Sort

```
void mergeSort(int v[], int inicio, int fim){
    int meio = floor((fim + inicio)/2);
    if(inicio < fim){
        //divisão
        mergeSort(v, inicio, meio);
        mergeSort(v, meio + 1, fim);

        //conquista
        merge(v, inicio, meio, fim);
    }
}
```

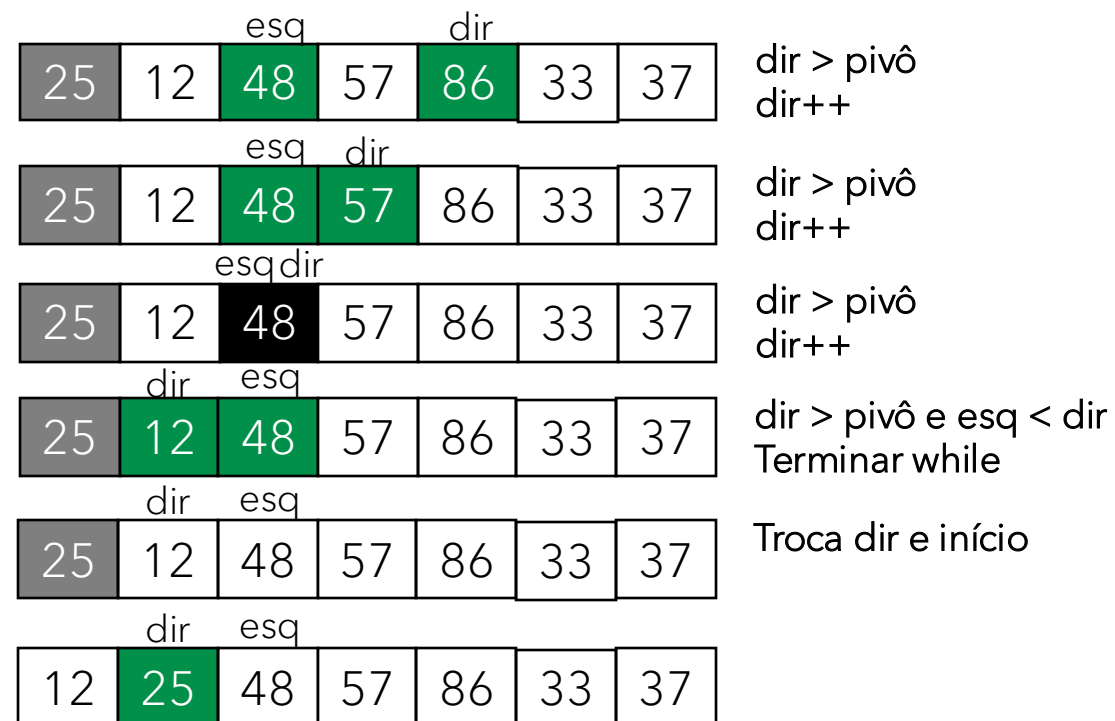
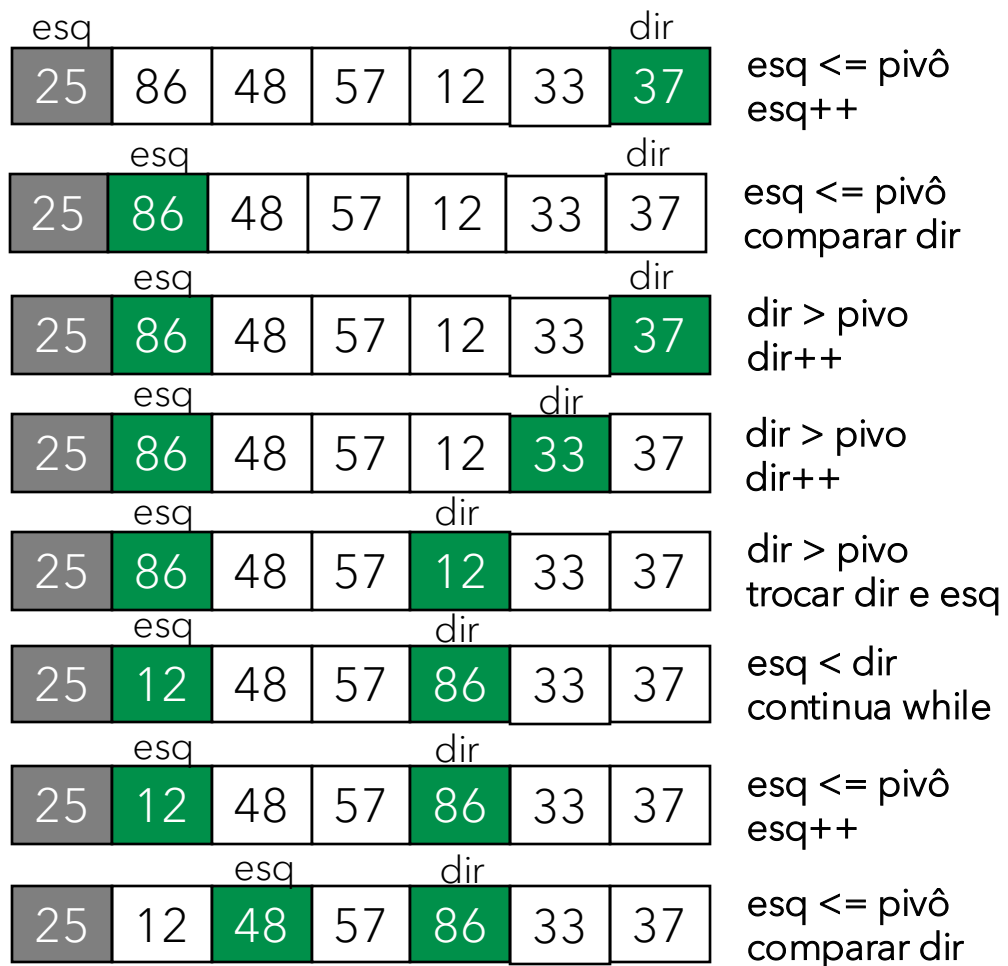
```
void merge(int v[], int inicio, int meio, int fim){
    int tamanho = fim + 1;
    int vAux[tamanho];
    int i = inicio; //início da primeira metade.
    int j = meio + 1; //início da segunda metade
    int k = 0;
    //intercala
    while(i <= meio && j <= fim){
        if(v[i] <= v[j]){
            vAux[k++] = v[i++];
            //igual a vAux[k] = v[i]; k++; i++;
        }else{
            vAux[k++] = v[j++];
        }
    }
    //copia o resto do subvetor
    while(i <= meio){
        vAux[k++] = v[i++];
    }
    while(j <= fim){
        vAux[k++] = v[j++];
    }
    //copia de volta para v
    for(i = inicio, k = 0; i <= fim; i++, k++){
        v[i] = vAux[k];
    }
}
```

Algoritmo *Quick Sort*

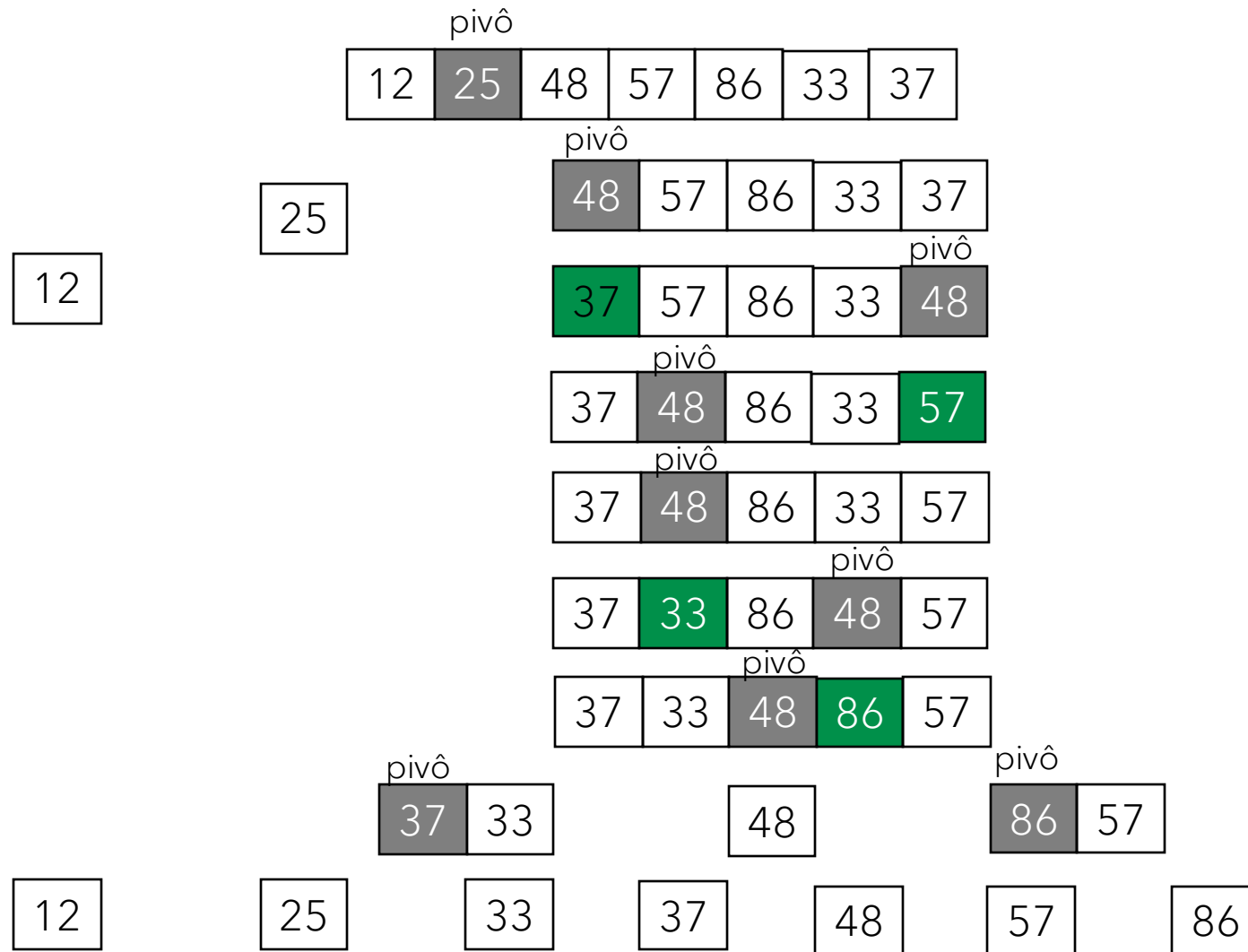
Algoritmo *Quick Sort*

- ▶ Também conhecido como ordenação por “partição”, é outro algoritmo recursivo que usa a ideia de “dividir para conquistar” para ordenar os dados de um *array*;
- ▶ O algoritmo, primeiramente, rearranja os valores de *array* de modo que os valores menores que o valor do **pivô** fiquem na parte esquerda do *array*, enquanto os valores maiores que o **pivô** ficam na parte direita;
- ▶ Supondo um *array* de tamanho **N** e que o **pivô** esteja na posição **X**, esse processo cria duas partições: $[0, \dots, X - 1]$ e $[X + 1, \dots, N - 1]$;
- ▶ Em seguida, o algoritmo é aplicado recursivamente a cada partição;
- ▶ Esse processo se repete até que cada partição contenha um único elemento.

Algoritmo Quick Sort



Algoritmo Quick Sort



Algoritmo *Quick Sort*

```
void quickSort(int *v, int inicio, int fim){
    int pivo;
    if(fim > inicio){
        pivo = particiona(v, inicio, fim);
        quickSort(v, inicio, pivo - 1);
        quickSort(v, pivo + 1, fim);
    }
}
```

```
int particiona(int *v, int inicio, int final){
    int esq, dir, pivo, aux;
    esq = inicio;
    dir = final;
    pivo = v[inicio];
    while(esq < dir){
        while(esq <= final && v[esq] <= pivo){
            esq++;
        }
        while(dir >= 0 && v[dir] > pivo){
            dir--;
        }
        if(esq < dir){
            aux = v[esq];
            v[esq] = v[dir];
            v[dir] = aux;
        }
    }
    v[inicio] = v[dir];
    v[dir] = pivo;
    return dir;
}
```